Visual Language Model API Redesign

Design Rationale

Why the current design needs changing?
The current VLMPipeline is monolithic — it owns the vision encoder, text embeddings, the tokenizer, and the LLM. The InputsEmbedder (which is the de facto "processor") is an internal class hidden in src, not exposed to users. This causes several problems:

- It is not possible to define target device and plugin config separately for the processor models and the LLM model. (**CVS-162621**)
- It is not possible to infer embeddings on NPU due to static shape limitations. (**CVS-178295**)
- Missing support for manual chat template application (with separate engine).
- No embedding reuse — you can't get vision embeddings without running the LLM.
- No cache reuse between text generation pipeline and embeddings-only workflows (retrieval, similarity search, etc.). Moving VisionRegistry from Pipeline to Processor allows sharing cached embeddings across multiple pipelines and use cases.
- Multiple methods to work with history — `start_chat()`/`finish_chat()` coexists with `ChatHistory`, causing confusion about which approach to use. (Note: `LLMPipeline` has already deprecated these methods; `VLMPipeline` has not yet.)
- Not aligned with other libraries, for example `transformers` — HuggingFace's pattern is Processor + Model, which allows users to inspect/modify embeddings between the two steps.

`transformers` canonical VLM flow:

```
processor = AutoProcessor.from_pretrained("model_id")
model = AutoModelForVision2Seq.from_pretrained("model_id")

inputs = processor(images=images, text=prompt, return_tensors="pt")
# → BatchFeature { input_ids, attention_mask, pixel_values }
# OR after internal vision encoding:
# → { inputs_embeds, attention_mask, position_ids }

output_ids = model.generate(**inputs)
text = processor.batch_decode(output_ids)
```

The main flow: the Processor produces a structured input; the model consumes it. The Processor can also be used standalone (e.g., get_image_features()).

Proposed API (C++ Headers)

1. VLMInputs — The structured intermediate

```cpp
// openvino/genai/visual_language/vlm_inputs.hpp

#pragma once
#include <optional>
#include <vector>
#include "openvino/runtime/tensor.hpp"

namespace ov::genai {

/// @brief Structured inputs for VLM generation, produced by VLMProcessor.
/// Analogous to HuggingFace's BatchFeature returned by a Processor.
struct VLMInputs {
    /// @brief Merged embeddings of text tokens and vision features.
    /// Shape: [1, sequence_length, hidden_size].
    /// The vision encoder outputs have already been projected into
    /// the LLM's embedding space and merged at <image>/<video> positions.
    ov::Tensor inputs_embeds;

    /// @brief Attention mask for the merged sequence.
    /// Shape: [1, sequence_length]. 1 = attend, 0 = ignore.
    ov::Tensor attention_mask;

    /// @brief Position IDs for the merged sequence.
    /// Always populated by VLMProcessor. For most models this is a
    /// linear range [history_size .. history_size + seq_len - 1].
    /// For Qwen2-VL / Qwen2.5-VL this contains 3D MROPE position IDs.
    /// Shape: [1, sequence_length] or model-specific (e.g., [3, 1, seq_len]).
    ov::Tensor position_ids;

    /// @brief Token type IDs (used by some models like Phi-4-MM and
    /// Gemma3), otherwise uninitialized.
    /// Shape: [1, sequence_length].
    ov::Tensor token_type_ids;

    /// @brief RoPE delta for position ID computation during autoregressive
    /// generation. Required by Qwen2-VL / Qwen2.5-VL (3D MROPE).
    /// For other models this remains nullopt and is not used.
    std::optional<int64_t> rope_delta;

    /// @brief Preprocessing timing (vision encoding + embedding merge).
    /// Populated by VLMProcessor::embed(); merged into VLMPerfMetrics
    /// by VLMPipeline::generate() so that a single VLMPerfMetrics
```

```
    /// covers the full request lifecycle.
    VLMRawPerfMetrics raw_perf_metrics;
};

} // namespace ov::genai
```

2. VLMProcessor — The public processor class

```
// openvino/genai/visual_language/processor.hpp

#pragma once
#include <filesystem>
#include <string>
#include <vector>
#include "openvino/runtime/tensor.hpp"
#include "openvino/genai/tokenizer.hpp"
#include "openvino/genai/visual_language/vlm_inputs.hpp"
#include "openvino/genai/visual_language/video_metadata.hpp"
#include "openvino/genai/visual_language/perf_metrics.hpp"
#include "openvino/genai/common_types.hpp"

namespace ov::genai {

/// @brief Per-model device assignment for the ModelsMap constructor.
/// Key is the model role name (same keys as ModelsMap, e.g.
/// "vision_embeddings", "text_embeddings", "resampler", ...).
/// Value is the device name ("CPU", "GPU", "NPU", etc.).
/// Models not listed default to "CPU".
using DeviceMapping = std::map<std::string, std::string>;

/// @brief A processor for Visual Language Models that handles
/// vision encoding, text tokenization, embedding merging, and
/// chat template application (via internal Tokenizer).
///
/// Analogous to transformers's AutoProcessor - combines an image
/// processor (VisionEncoder) and a tokenizer/embedder into a
/// single preprocessing pipeline.
///
/// Usage:
///   auto processor = VLMProcessor("path/to/models", "GPU");
///   auto inputs = processor.embed("Describe this image <ov_image_0>", {image_tensor});
///   auto result = llm.generate(inputs, generation_config);
class OPENVINO_GENAI_EXPORTS VLMProcessor {
public:
    /// @brief Construct a processor from a directory containing
    /// vision encoder, text embeddings model, tokenizer, and configs.
```

```
/// @param models_path Directory with openvino_vision_embeddings_model.xml,
///        openvino_text_embeddings_model.xml, tokenizer files, and config.json.
/// @param device Inference device for vision encoder and embeddings model.
/// @param properties Device configuration properties.
VLMProcessor(
    const std::filesystem::path& models_path,
    const std::string& device,
    const ov::AnyMap& properties = {}
);

/// @brief Construct from pre-loaded models.
/// @param models_map Map of model name to (IR string, weights tensor) pairs.
///        Required keys: "vision_embeddings", "text_embeddings".
///        Model-specific optional keys:
///        - "resampler" (MiniCPM, LLaVA Next Video)
///        - "vision_embeddings_merger" (Qwen2-VL, Qwen2.5-VL)
///        - "vision_projection" (Phi3-V, Phi4-MM)
///        - "multi_modal_projector" (LLaVA Next Video)
/// @param tokenizer Pre-initialized tokenizer.
/// @param config_dir_path Path to directory containing config.json.
/// @param device_mapping Inference device per model type, e.g.:
///        {{"vision_embeddings", "NPU"}, {"text_embeddings", "CPU"}}.
///        By default all models run on CPU.
/// @param properties Per model device configuration properties, e.g.:
///        {"PER_MODEL_PROPERTIES":
///          {"vision_embeddings", {{"NUM_STREAMS", "8"}}},  # Causing InferReq queue s
///          {"text_embeddings",   {{"NUM_STREAMS", "4"}}}}}.# Causing InferReq queue s
///        Properties from top-level are also applied to all models
///        unless overridden in PER_MODEL_PROPERTIES.
///        DEVICE_PROPERTIES can be used to specify device-specific properties, e.g.:
///        {"DEVICE_PROPERTIES": {"NPU", {{"NPUW_SPECIFIC_PROPERTY", "XXX"}}}}}.
///
/// Usage:
///   ModelsMap models_map = {
///     {"vision_embeddings", {vision_ir_str, vision_weights_tensor}},
///     {"text_embeddings", {text_ir_str, text_weights_tensor}}
///   };
///   DeviceMapping device_mapping = {
///     {"vision_embeddings", "NPU"},
///     {"text_embeddings", "CPU"}
///   };
///   ov::AnyMap properties = {
///     {"PER_MODEL_PROPERTIES", {
///         {"vision_embeddings", {{"NUM_STREAMS", "8"}}},  # Causing InferReq queue si
///         {"text_embeddings",   {{"NUM_STREAMS", "4"}}}}. # Causing InferReq queue si
///     }},
```

```
//      {"DEVICE_PROPERTIES", {
///         {"NPU", {{"NPUW_SPECIFIC_PROPERTY", "XXX"}}}}}
///     {"GLOBAL_PROPERTY", "value"} // applied to all models unless overridden
///   };
///   auto processor = VLMProcessor(
///       models_map,
///       tokenizer,
///       config_dir_path,
///       device_mapping,
///       properties);
VLMProcessor(
    const ModelsMap& models_map,
    const Tokenizer& tokenizer,
    const std::filesystem::path& config_dir_path,
    const DeviceMapping& device_mapping = {},
    const ov::AnyMap& properties = {}
);

~VLMProcessor();

/// @brief Prepare inputs for VLM generation from a text prompt
/// and optional images/videos.
///
/// Performs the preprocessing pipeline:
/// 1. DOES NOT apply chat template - prompt is treated as a
///    pre-formatted string. Use the overload with ChatHistory for
///    automatic template application.
/// 2. Normalizes image/video tags (universal <-> model-native).
/// 3. Encodes images/videos through the vision encoder.
///    NOTE: this overload uses VisionRegistry caching -
///    therefore VisionRegistry ownership must be moved from
///    VLMPipeline/ChatHistory to VLMProcessor.
/// 4. Tokenizes the text and computes text embeddings.
/// 5. Merges vision features into the text embedding sequence
///    at <image>/<video> placeholder positions.
/// 6. Computes attention mask, position IDs, and (if needed)
///    token type IDs.
///
/// For using image and video tags in prompt, see:
/// https://openvinotoolkit.github.io/openvino.genai/docs/use-cases/image-processing/
///
/// @param prompt Text prompt, prepared manually by the caller.
/// @param images RGB image tensors with [NHWC] or [HWC] layout.
/// @param videos Video frame tensors with [NHWC] layout.
/// @param videos_metadata Optional per-video metadata controlling
///        frame sampling.
```

```
/// @return VLMInputs ready to pass to VLMPipeline::generate()
///         or ContinuousBatchingPipeline::add_request().
VLMInputs embed(
    const std::string& prompt,
    const std::vector<ov::Tensor>& images = {},
    const std::vector<ov::Tensor>& videos = {},
    const std::vector<VideoMetadata>& videos_metadata = {}
);

/// @brief Overload that accepts a ChatHistory.
/// Applies the chat template internally via
/// get_tokenizer().apply_chat_template(), then follows the same
/// encoding, tokenizing, merging, and position-computation steps
/// as the string overload.
///
/// @param history ChatHistory containing system, user, and assistant turns.
/// @param images Optional image tensors.
/// @param videos Optional video tensors.
/// @param videos_metadata Optional per-video metadata.
/// @return VLMInputs with the chat template applied to the history
///         and vision features merged in.
VLMInputs embed(
    const ChatHistory& history,
    const std::vector<ov::Tensor>& images = {},
    const std::vector<ov::Tensor>& videos = {},
    const std::vector<VideoMetadata>& videos_metadata = {}
);

/// @brief Extract image embeddings without merging with text.
/// Useful for embedding-only workflows (retrieval, caching,
/// analysis) without invoking the LLM.
///
/// Analogous to HuggingFace's model.get_image_features().
///
/// @param image A single image as a 4D tensor with [NHWC] layout,
///        where N=1. Shape: [1, H, W, C].
/// @return Embedding tensor. Shape: [num_patches, hidden_size].
ov::Tensor get_image_embeddings(const ov::Tensor& image);

/// @brief Extract video embeddings without merging with text.
/// Useful for embedding-only workflows (retrieval, caching,
/// analysis) without invoking the LLM.
///
/// @param video Video frames as a 4D tensor with [NHWC] layout,
///        where N is the number of frames. Shape: [N, H, W, C].
/// @param video_metadata Optional metadata for frame sampling.
```

```
    /// @return Embedding tensor. Shape: [num_tokens, hidden_size].
    ov::Tensor get_video_embeddings(
        const ov::Tensor& video,
        const VideoMetadata& video_metadata = {}
    );

    // get_text_embeddings()?

    /// @brief Get the underlying tokenizer.
    /// Also provides access to set_chat_template() for overriding
    /// the chat template used by embed(ChatHistory, ...).
    Tokenizer get_tokenizer() const;
};

} // namespace ov::genai
```

3. Redesigned VLMPipeline — LLM-only

```
// openvino/genai/visual_language/pipeline.hpp (redesigned)

#pragma once
#include <filesystem>
#include <string>
#include "openvino/genai/generation_config.hpp"
#include "openvino/genai/streamer_base.hpp"
#include "openvino/genai/tokenizer.hpp"
#include "openvino/genai/visual_language/vlm_inputs.hpp"
#include "openvino/genai/visual_language/perf_metrics.hpp"
#include "openvino/genai/common_types.hpp"

namespace ov::genai {

/// @brief A Visual Language Model pipeline that performs text generation
/// given pre-processed VLMInputs from a VLMProcessor.
///
/// This class owns only the LLM (language model). Vision encoding and
/// embedding preparation are handled by VLMProcessor.
///
/// Usage:
///   auto processor = VLMProcessor("path/to/models", "GPU");
///   auto llm = VLMPipeline("path/to/models", processor, "GPU");
///   auto inputs = processor.embed(
///         "Describe this image <ov_image_0>", {image_tensor});
///   auto result = llm.generate(inputs, generation_config);
class OPENVINO_GENAI_EXPORTS VLMPipeline {
public:
```

```
/// @brief Construct the LLM pipeline from a model directory.
/// @param models_path Path to directory containing the language model IR
///        (openvino_language_model.xml) and generation_config.json.
///        This is typically the same directory as the one passed to
///        VLMProcessor, which loads vision/embeddings models from it.
/// @param processor A VLMProcessor instance, responsible for vision encoding,
///        embedding preparation, and tokenization.
/// @param device Inference device.
/// @param properties Device configuration properties.
VLMPipeline(
    const std::filesystem::path& models_path,
    const VLMProcessor& processor,
    const std::string& device,
    const ov::AnyMap& properties = {}
);

/// @brief Construct from a pre-loaded language model.
/// @param model_str Language model IR as string.
/// @param weights_tensor Model weights.
/// @param processor A VLMProcessor instance.
/// @param config_dir_path Path to directory containing generation_config.json.
/// @param device Inference device.
/// @param properties Device configuration properties.
VLMPipeline(
    const std::string& model_str,
    const ov::Tensor& weights_tensor,
    const VLMProcessor& processor,
    const std::filesystem::path& config_dir_path,
    const std::string& device,
    const ov::AnyMap& properties = {}
);

~VLMPipeline();

/// @brief Generate text from pre-processed VLMInputs.
/// @param inputs Structured inputs from VLMProcessor::embed().
/// @param generation_config Text generation parameters.
/// @param streamer Optional streamer for token-by-token output.
/// @return Generated text(s) with scores and performance metrics.
///         VLMDecodedResults::perf_metrics includes timing from
///         both VLMProcessor::embed() (via inputs.raw_perf_metrics)
///         and the LLM generation phase.
VLMDecodedResults generate(
    const VLMInputs& inputs,
    const GenerationConfig& generation_config,
    const StreamerVariant& streamer = std::monostate{}
```

```
);

/// @brief Generate with config as property map (for Python bindings convenience).
VLMDecodedResults generate(
    const VLMInputs& inputs,
    const ov::AnyMap& config_map
);

/// ---- Legacy convenience overloads (delegate to internal processor) ----
/// These use VLMProcessor passed in constructor for backward compatibility.
/// New code should prefer the VLMProcessor + generate(VLMInputs) pattern.
/// Marked [[deprecated]] and scheduled for removal in the next major release.

/// @deprecated Use VLMProcessor::embed() + generate(VLMInputs) instead.
[[deprecated("Use VLMProcessor::embed() + generate(VLMInputs) instead.")]]
VLMDecodedResults generate(
    const std::string& prompt,
    const std::vector<ov::Tensor>& images,
    const GenerationConfig& generation_config,
    const StreamerVariant& streamer = std::monostate{}
);

/// @deprecated Use VLMProcessor::embed() + generate(VLMInputs) instead.
[[deprecated("Use VLMProcessor::embed() + generate(VLMInputs) instead.")]]
VLMDecodedResults generate(
    const std::string& prompt,
    const ov::AnyMap& config_map
);

/// @brief Get the tokenizer (for output detokenization if needed).
Tokenizer get_tokenizer() const;

GenerationConfig get_generation_config() const;
void set_generation_config(const GenerationConfig& config);

/// ---- Backward-compatible constructors ----
/// These create a VLMProcessor internally from the same arguments.
/// Existing user code continues to work without changes.

/// @brief Convenience constructor - creates VLMProcessor internally.
VLMPipeline(
    const std::filesystem::path& models_path,
    const std::string& device,
    const ov::AnyMap& properties = {}
);
```

```
    /// @brief Convenience constructor from pre-loaded models -
    /// creates VLMProcessor internally.
    VLMPipeline(
        const ModelsMap& models_map,
        const Tokenizer& tokenizer,
        const std::filesystem::path& config_dir_path,
        const std::string& device,
        const ov::AnyMap& properties = {},
        const GenerationConfig& generation_config = {}
    );
};

} // namespace ov::genai
```

4. ContinuousBatchingPipeline — VLM-aware add_request()

```
// In continuous_batching_pipeline.hpp - additions to existing class.
// All existing VLM overloads (accepting raw images/videos) remain unchanged.

// New add_request overload accepting VLMInputs:

/// @brief Add a VLM request with pre-processed inputs from VLMProcessor.
/// @param request_id Unique request identifier.
/// @param inputs Pre-processed VLMInputs containing merged embeddings.
/// @param sampling_params Generation configuration.
/// @return Handle to monitor and read generation results.
GenerationHandle add_request(
    uint64_t request_id,
    const VLMInputs& inputs,
    const ov::genai::GenerationConfig& sampling_params
);

// New generate overload for batch VLM:

/// @brief Batch generate from pre-processed VLMInputs.
std::vector<VLMDecodedResults> generate(
    const std::vector<VLMInputs>& inputs,
    const std::vector<GenerationConfig>& sampling_params,
    const StreamerVariant& streamer = std::monostate{}
);
```

Complete usage examples

Example 1: VLMPipeline — basic single image

```
// Separate construction: processor owns vision models, pipeline owns LLM
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
```

```cpp
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

// Embed: encodes images, tokenizes text, merges embeddings
auto inputs = processor.embed(
    "<|im_start|><ov_image_0> Describe this image<|im_end|><|im_start|>", {image_tensor});

// Generate: runs LLM on pre-merged embeddings
ov::genai::GenerationConfig config;
config.max_new_tokens = 256;
auto result = llm.generate(inputs, config);
std::cout << result.texts.at(0) << std::endl;
```

Example 1a: VLMPipeline — basic single image (Python)

```python
import openvino_genai as ov_genai

processor = ov_genai.VLMProcessor("path/to/models", "GPU")
llm = ov_genai.VLMPipeline("path/to/models", processor, "GPU")

inputs = processor.embed(
    "<|im_start|><ov_image_0> Describe this image<|im_end|><|im_start|>",
    images=[image_tensor],
)

result = llm.generate(inputs, ov_genai.GenerationConfig(max_new_tokens=256))
print(result.texts[0])
```

Example 2: Custom device mapping and plugin configuration

```cpp
// Load models from memory (e.g., read from custom storage or simply from disk)
ov::genai::ModelsMap models_map = {
    {"vision_embeddings", {vision_ir_str, vision_weights_tensor}},
    {"text_embeddings", {text_ir_str, text_weights_tensor}},
    {"resampler", {resampler_ir_str, resampler_weights_tensor}}
};

// Assign each model to a different device
ov::genai::DeviceMapping device_mapping = {
    {"vision_embeddings", "NPU"},
    {"text_embeddings", "CPU"},
    {"resampler", "GPU"}
};

// Properties: per-model, per-device, and global
ov::AnyMap properties = {
    // Per-model properties override global ones for the specified model
```

```
    {"PER_MODEL_PROPERTIES", ov::AnyMap{
        {"vision_embeddings", ov::AnyMap{{"NUM_STREAMS", "8"}}},// Makes InferReq queue size
        {"text_embeddings",   ov::AnyMap{{"NUM_STREAMS", "4"}}} // Makes InferReq queue size
    }},
    // Per-device properties apply to all models on that device
    {"DEVICE_PROPERTIES", ov::AnyMap{
        {"NPU", ov::AnyMap{{"NPUW_SPECIFIC_PROPERTY", "XXX"}}},
        {"GPU", ov::AnyMap{{"GPU_QUEUE_PRIORITY", "HIGH"}}}
    }},
    // Global property - applied to all models unless overridden above
    {"CACHE_DIR", "/tmp/ov_cache"}
};

ov::genai::Tokenizer tokenizer("path/to/tokenizer");

auto processor = ov::genai::VLMProcessor(
    models_map,
    tokenizer,
    "path/to/config_dir",  // contains image/video preprocessing settings
    device_mapping,
    properties
);

// LLM runs on GPU with its own properties if needed
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

auto inputs = processor.embed(
    "<|im_start|><ov_image_0> Describe this image<|im_end|><|im_start|>", {image_tensor});
auto result = llm.generate(inputs, config);
```

Example 3: Multi-turn chat with multiple images via ChatHistory

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

// Build a ChatHistory with multipart content (OpenAI-like format)
// Possible to add tool definitions and extra args as well if needed
ov::genai::ChatHistory history = {
    {{"role", "system"}, {"content", "You are a helpful assistant."}},
    {{"role", "user"}, {"content", {
        {{"type", "text"}, {"text", "Compare these two images:"}},
        {{"type", "image"}},
        {{"type", "image"}}
    }}}
};
```

```
// embed() applies the chat template and merges vision features
auto inputs = processor.embed(history, {image1, image2});
auto result = llm.generate(inputs, config);
```

Example 4: Embedding extraction (no LLM)

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");

// Extract image embeddings - each image is a 4D tensor [1, H, W, C]
auto img_emb1 = processor.get_image_embeddings(image1);  // [num_patches, hidden_size]
auto img_emb2 = processor.get_image_embeddings(image2);

// Extract video embeddings - video is a 4D tensor [N_frames, H, W, C]
auto vid_emb = processor.get_video_embeddings(video1);    // [num_tokens, hidden_size]

// Extract video embeddings with metadata-driven sampling
ov::genai::VideoMetadata meta;
meta.total_num_frames = 120;
meta.fps = 24.0f;
meta.frames_indices = {0, 15, 30, 45, 60, 75, 90, 105};
auto vid_emb_sampled = processor.get_video_embeddings(video1, meta);

// Use for retrieval, similarity, caching, etc.
```

Example 5: ContinuousBatchingPipeline — concurrent VLM requests

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::ContinuousBatchingPipeline("path/to/models", scheduler_config, process

// Embed multiple requests
auto inputs1 = processor.embed(
    "<|im_start|><ov_image_0> What is in this photo?<|im_end|><|im_start|>", {photo1});

auto inputs2 = processor.embed(
    "<|im_start|><ov_video_0> Describe this video<|im_end|><|im_start|>",
    {},          // no images
    {video}
);

// Submit to continuous batching
auto handle1 = llm.add_request(1, inputs1, config);
auto handle2 = llm.add_request(2, inputs2, config);

// Process until done
while (llm.has_non_finished_requests()) {
    llm.step();
```

```
}

auto results1 = handle1->read_all();
auto results2 = handle2->read_all();
```

Example 6: Video with metadata — VLMPipeline

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

// Prepare per-video metadata
ov::genai::VideoMetadata meta;
meta.total_num_frames = 120;
meta.fps = 24.0f;
// Explicitly select 8 frames: pipeline skips model-specific sampling
meta.frames_indices = {0, 15, 30, 45, 60, 75, 90, 105};

auto inputs = processor.embed(
    "<|im_start|><ov_video_0> Describe this video<|im_end|><|im_start|>",
    {},             // no images
    {video_tensor}, // video with all 120 frames
    {meta}          // per-video metadata
);

ov::genai::GenerationConfig config;
config.max_new_tokens = 256;
auto result = llm.generate(inputs, config);
std::cout << result.texts.at(0) << std::endl;
```

Example 8: Inspecting / modifying embeddings between processor and LLM

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

auto inputs = processor.embed(
    "<|im_start|><ov_image_0> Describe this image<|im_end|><|im_start|>", {image_tensor});

// User can inspect or transform embeddings before generation
std::cout << "Embedding sequence length: "
          << inputs.inputs_embeds.get_shape()[1] << std::endl;

// For example, truncate if too long
// ... modify inputs.inputs_embeds, inputs.attention_mask ...

auto result = llm.generate(inputs, config);
```

Design Decisions

Naming: `embed()`, `get_image_embeddings()`, `get_video_embeddings()`

While `embed()` performs more than embedding (tag normalization, vision encoding, tokenization, merging, position computation), the name communicates the purpose to the caller: "produce embeddings for the LLM." The term mimics the transformers pattern where the processor callable returns `BatchFeature`:

GenAI proposal:

```
inputs = processor.embed(prompt, images)  # VLMInputs { inputs_embeds, attention_mask, posit
```

Transformers library:

```
inputs = processor(images=images, text=prompt, return_tensors="pt") # BatchFeature { input_i
```

Vision embedding extraction is split into `get_image_embeddings()` and `get_video_embeddings()` rather than a single `get_vision_embeddings()` that accepts a batch vector. Both methods take a single 4D `ov::Tensor` with `[N, H, W, C]` layout — `N=1` for images, `N>1` (number of frames) for videos. This avoids batch-vector ambiguity (the caller would otherwise need to ensure correct image/video ordering and the encoder would need to guess which inputs are images vs videos). Separate make the intent explicit, and also extensible for future modalities such as audio.

Multi-Turn Chat via ChatHistory

Drop `start_chat()` / `finish_chat()` from `VLMPipeline`. `LLMPipeline` has already deprecated both methods (PR #3217); `VLMPipeline` still has them undeprecated. The new design removes them entirely. Multi-turn chat is handled exclusively through `ChatHistory`:

- The caller manages the conversation via `ChatHistory` (appending messages, passing images).
- `VLMProcessor::embed(ChatHistory, ...)` applies the chat template and produces `VLMInputs`.
- The pipeline receives `VLMInputs` and manages KV cache internally.

Chat template modifications

To override the chat template, call `processor.get_tokenizer().set_chat_template(...)` — no dedicated method on `VLMProcessor` is needed.

Processor manages Embeddings cache via Vision Registry

The processor is able to **cache** embeddings — `VLMProcessor::embed()` always produces embeddings for the full prompt or full `ChatHistory`. It does not

track what the pipeline has already processed, other than full image/video deduplication via the shared `VisionRegistry`. This means that if the same image appears in multiple calls to `embed()`, it will be encoded once and cached in the `VisionRegistry` for subsequent reuse, regardless of how many pipelines or requests are using it.

The existing `VLMChatContext` logic for `needs_kv_cache_reset` moves entirely into the pipeline implementation. The `VisionRegistry` (owned by the processor but shared with the pipeline) continues to deduplicate encoded images across turns — the processor checks the registry before re-encoding an image, regardless of KV cache state.

VisionRegistry Ownership and Sharing

`VisionRegistry` stays an internal class — a cache storage for image/videos with corresponding embeddings. In the current codebase it is owned by each pipeline.

In the new design, `VisionRegistry` ownership moves to `VLMProcessor::Impl`. The pipeline needs access to the registry (for KV cache coordination and chat context processing) but does not expose it to users.

```
// In pipeline.cpp (internal implementation)
VLMPipeline::Impl::Impl(const VLMProcessor& processor, ...)
    : m_vision_registry(processor.m_impl->get_vision_registry()),  // shared_ptr copy
      ... {}
```

Model-Specific InputsEmbedder Subclasses

The 11 model-specific subclasses (at the moment of writing this document) (`InputsEmbedderMiniCPM`, `InputsEmbedderLLaVA`, `InputsEmbedderLLaVANext`, `InputsEmbedderLLaVANextVideo`, `InputsEmbedderNanoLLaVA`, `InputsEmbedderInternVLChat`, `InputsEmbedderPhi3V`, `InputsEmbedderPhi4MM`, `InputsEmbedderQwen2VL`, `InputsEmbedderQwen2_5_VL`, `InputsEmbedderGemma3`) move from the pipeline to the processor. No public API change needed.

Thread Safety, Concurrency

All three internal components of `VLMProcessor` support concurrent access:

- **`VisionEncoder`** — uses `CircularBufferQueue<ov::InferRequest>` (queue size depending on `ov::optimal_number_of_infer_requests` which is affected by `NUM_STREAMS` from plugin config). NUM_STREAMS can be set globally for the entire processor via top-level properties, or individually per model via `PER_MODEL_PROPERTIES` in the constructor.
- **`EmbeddingsModel`** — uses `CircularBufferQueue<EmbeddingsRequest>` with the same pattern.
- **`VisionRegistry`** — guarded by `mutable std::mutex m_mutex` in all public methods.

Therefore `VLMProcessor::embed()` is **thread-safe** and can be called concurrently from multiple pipeline instances or threads without external synchronization.

`VLMProcessor::generate` however, should remain non-thread-safe and should not be used from multiple threads. (the same as `ContinuousBatchingPipeline::generate`)

Continuous Batching Pipeline's `add_request()` remains thread safe and ready to use for concurrent usage.

NPU Considerations

Processor-side (VisionEncoder on NPU):

NPU requires static input shapes at compile time. Vision encoder support depends on the model category:

1. **Fixed-crop models** (LLaVA, InternVL, Gemma3, NanoLLaVA): image input is always resized to `config.crop_size_height × config.crop_size_width` (e.g., 384×384), producing a deterministic `pixel_values` shape. These can run on NPU by reshaping the model to the known fixed dimensions at compile time. For that reason, NPU could consume plugin config property:

```
ov::AnyMap properties = {
    {"PER_MODEL_PROPERTIES", ov::AnyMap{
        {"vision_encoder", ov::AnyMap{
            {"WIDTH", "336"},
            {"HEIGHT", "224"},
        }}
    }}
};
```

2. **Variable-shape models** (Qwen2VL, MiniCPM, Phi3V, Phi4MM): image dimensions vary based on "smart_resize" logic. NPU support would also require setting static width and height via property, but it would be calculated in advance based on the model's patch/grid logic.

Text Embeddings Model on NPU

The same as above, but with MAX_PROMPT_LEN and padding.

PER_MODEL_PROPERTIES extraction

The design describes three ways to define property (priority order from lowest to highest): - global (top-level) as usual - DEVICE_PROPERTIES (**it exists now**) -> applies to all models on that device, overrides global properties for those models - PER_MODEL_PROPERTIES (**new**) -> applies to a specific model, overrides both global and device properties

Backward Compatibility

VLMPipeline

Constructors: Both existing `VLMPipeline` constructors are kept as non-deprecated convenience overloads that create a `VLMProcessor` internally. They construct a `VLMProcessor` and delegate to `VLMPipeline(models_path, processor, device, properties)`. Existing user code continues to work without changes.

Legacy `generate()` overloads: Overloads that accept raw prompts/images (without `VLMInputs`) carry `[[deprecated]]` attributes directing users to the `VLMProcessor::embed()` + `generate(VLMInputs)` pattern. Scheduled for removal in the next major release.

ContinuousBatchingPipeline

`ContinuousBatchingPipeline` serves both text-only and VLM workloads. For text-only overloads, no changes are needed - internal InputsEmbedder will be created automatically. To work with VLM inputs, construction should provide a `VLMProcessor` instance, and users should call the new `add_request(request_id, VLMInputs, GenerationConfig)` overload that accepts pre-processed inputs. Existing `add_request()` overloads remain unchanged and non-deprecated for backward compatibility.

Implementation Steps

Phase 1: Add VLMProcessor and VLMInputs (new public classes, possible duplicated code)

Classes `VLMProcessor`, `VLMInputs`, `DeviceMapping` as new public API. No changes to existing classes.

- Add `VLMInputs` struct (new headers under `visual_language/`).
- Add `DeviceMapping` typedef in `common_types.hpp`.
- Implement `VLMProcessor` — internally takes ownership of `VisionEncoder`, `EmbeddingsModel`, `VisionRegistry`, and `IInputsEmbedder` (moved from `VLMPipelineImpl`; source files stay in place).
- Implement `PER_MODEL_PROPERTIES` resolution (global → `DEVICE_PROPERTIES` → `PER_MODEL_PROPERTIES`).
- Add Python bindings, unit tests, docs.

Deliverable: `VLMProcessor` works standalone. `VLMPipeline` unchanged.

Phase 2: Use VLMProcessor/VLMInputs in VLMPipeline and ContinuousBatchingPipeline (breaking change with deprecation)

`VLMPipeline` and `ContinuousBatchingPipeline` consumes `VLMInputs` as primary input; old API deprecated.

- Add `VLMPipeline(path, processor, device)` constructors — pipeline loads only the LLM, shares `VisionRegistry` with the processor.
- Add `generate(VLMInputs, ...)` overloads.
- Deprecate `generate(prompt, images, ...)`, `start_chat()`, `finish_chat()` with `[[deprecated]]`.
- Legacy constructors (`VLMPipeline(path, device)`) create a `VLMProcessor` internally — not deprecated.
- Add `ContinuousBatchingPipeline` constructor accepting `VLMProcessor`.
- Add `add_request(request_id, VLMInputs, config)` and batch `generate(vector<VLMInputs>, ...)` overloads.
- Existing `add_request(prompt, images, ...)` overloads unchanged.
- Update Python bindings, samples, tests.

Deliverable: Recommended flow is `processor.embed()` → `pipeline.generate(VLMInputs)`.